iroh-blob 동시성 및 비동기 처리 분석

개요

iroh-blob은 고성능 데이터 전송을 위해 tokio 기반의 비동기 런타임을 활용하여 복잡한 동시성 제어 메커니즘을 구현합니다. 이 문서는 시스템의 비동기 처리 아키텍처, 태스크 관리 방식, 엔티티별 동시성 제어, 그리고 백프레셔 및 플로우 컨트롤 메커니즘을 상세히 분석합니다.

tokio 기반 비동기 런타임 활용

비동기 런타임 구조

iroh-blob은 tokio를 기반으로 한 완전한 비동기 아키텍처를 채택하고 있습니다:

```
// 메인 액터 실행 패턴

tokio::spawn(actor.run());



// 비동기 선택적 처리

tokio::select! {

biased;

msg = self.rx.recv() => {

if  let Some(msg) = msg {

self.handle_msg(msg).await;

} else {

break;
```

```
}
}
// 다른 이벤트들...
}
```

비동기 I/O 최적화

시스템은 다양한 레벨에서 비동기 I/O를 최적화합니다:

1. **버퍼링된 읽기**: 대용량 파일 처리 시 버퍼 크기를 동적으로 조정
2. **스트리밍 처리**: 메모리 사용량을 제한하면서 대용량 데이터 처리
3. **청크 단위 처리**: 1024바이트 청크 단위로 데이터를 분할하여 처리

```
// 버퍼 크기 최적화 예시
let size = usize::try_from(outboard.tree.size()).unwrap_or(usize::MAX);
let read_buf_size = (64 * 1024).min(size);
let chunk_buf_size = size.min(outboard.tree.block_size().bytes());
let reader = BufReader::with_capacity(read_buf_size, data);
```

JoinSet을 이용한 태스크 관리

JoinSet 활용 패턴

iroh-blob은 여러 모듈에서 `JoinSet` 을 사용하여 동시 실행되는 태스크들을 효율적으로 관리합니다:

1. 파일 시스템 저장소 (FsStore)

```
struct Actor {
commands: tokio::sync::mpsc::Receiver<InternalCommand>,
tasks: JoinSet<()>, // 가져오기/내보내기 작업용 태스크 집합
handles: EntityManagerState<EmParams>, // 엔티티별 동시성 제어
}
```

2. 메모리 저장소 (MemStore)

```
struct Actor {
commands: tokio::sync::mpsc::Receiver<Command>,
tasks: JoinSet<TaskResult>, // 다양한 작업 결과를 처리하는 태스크 집합
state: State,
}
```

3. 다운로더 (Downloader)

```
struct DownloaderActor {
store: Store,
pool: ConnectionPool,
tasks: JoinSet<()>, // 다운로드 태스크 관리
running: HashSet<tokio::task::Id>, // 실행 중인 태스크 추적
}
```

태스크 생명주기 관리

태스크 생성과 추적

```
impl DownloaderActor {
fn spawn(&mut self, fut: impl Future<Output = ()> + Send + 'static) {
let span = tracing::Span::current();
let id = self.tasks.spawn(fut.instrument(span)).id();
self.running.insert(id); // 실행 중인 태스크 ID 추적
}
}
```

태스크 완료 처리

```
// 태스크 완료 대기 및 오류 처리
while let Some(res) = tasks.join_next().await {
if let Err(e) = res {
error!("Task failed: {e:?}");
}
}
```

우아한 종료 (Graceful Shutdown)

시스템은 두 가지 종료 방식을 지원합니다:

1. 소프트 종료 (Soft Shutdown)

```
async fn soft_shutdown(self) {
let Self { mut tasks, state, .. } = self;
state.shutdown().await; // 상태 정리
while let Some(res) = tasks.join_next().await {
if let Err(e) = res {
eprintln!("Task failed during shutdown: {e:?}");
}
}
}
```

2. 하드 종료 (Hard Shutdown)

```
async fn hard_shutdown(self) {
let Self { mut tasks, state, .. } = self;
drop(state); // 즉시 상태 해제
while let Some(res) = tasks.join_next().await {
if let Err(e) = res {
eprintln!("Task failed during shutdown: {e:?}");
}
}
}
```

엔티티 매니저를 통한 해시별 동시성 제어

엔티티 매니저 아키텍처

엔티티 매니저는 iroh-blob의 핵심 동시성 제어 메커니즘으로, 해시별로 독립적인 동시성을 제공하면서 단일 해시 내에서는 순차적 처리를 보장합니다.

```
/// 엔티티 간 병렬성을 제공하지만 단일 엔티티 내에서는 동시성만 제공
/// 파일과 같은 외부 리소스를 래핑하는 엔티티에 유용
pub struct EntityManager<P: Params>(mpsc::Sender<main_actor::Command<P>>);
```

동시성 제어 원칙

1. **엔티티 간 병렬성**: 서로 다른 해시에 대한 작업은 완전히 병렬로 실행
2. **엔티티 내 순차성**: 동일한 해시에 대한 작업은 순차적으로 실행
3. **리소스 보호**: 파일 핸들과 같은 외부 리소스에 대한 안전한 접근 보장

엔티티 상태 관리

```
pub trait Params: Send + Sync + 'static {
type EntityId: Debug + Hash + Eq + Clone + Send + Sync + 'static;
type GlobalState: Debug + Clone + Send + Sync + 'static;
type EntityState: Default + Debug + Reset + Clone + Send + Sync + 'static;
fn on_shutdown(
state: entity_actor::State<Self>,
cause: ShutdownCause,
) -> impl Future<Output = ()> + Send + 'static;
```

```
}
```

태스크 스폰 메커니즘

```
pub  async  fn  spawn<F, Fut>(&self, id: P::EntityId, f: F) -> Result<(), &'stat
where
F: FnOnce(SpawnArg<P>) -> Fut + Send + 'static,
Fut: future::Future<Output = ()> + Send + 'static,
{
let  spawn = Spawn {
id,
f: Box::new(|arg| {
Box::pin(async  move {
f(arg).await;
})
}),
};
self.0
.send(main_actor::Command::Spawn(spawn))
.await
.map_err(|_| "Failed to send spawn command")
}
```

엔티티 액터 풀링

시스템은 성능 최적화를 위해 엔티티 액터를 재사용합니다:

```
pub struct Options {
pub pool_capacity: usize, // 재사용할 비활성 액터 수
pub inbox_size: usize, // 매니저 액터 인박스 크기
pub entity_inbox_size: usize, // 엔티티 액터 인박스 크기
pub entity_response_inbox_size: usize, // 응답 인박스 크기
pub entity_futures_initial_capacity: usize, // 퓨처 집합 초기 용량
}
```

백프레셔와 플로우 컨트롤 구현

채널 기반 백프레셔

iroh-blob은 tokio의 mpsc 채널을 활용하여 자연스러운 백프레셔를 구현합니다:

1. 채널 용량 제한

```
// 제한된 용량의 채널로 백프레셔 구현
let (send, recv) = mpsc::channel(options.inbox_size);
```

2. 송신 실패 처리

```
if let Err(TokioSendError(e)) = conn_tx.send(msg).await {
// 채널이 가득 찬 경우 또는 수신자가 종료된 경우 처리
```

```
msg = e;
}
```

버퍼링 전략

1. 동적 버퍼 크기 조정

```
fn get_buffer_size(size: NonZeroU64) -> usize {
(size.get() / (IROH_BLOCK_SIZE.bytes() as u64) + 2).min(64) as usize
}
```

2. 스트림 버퍼링

```
// 제한된 동시성으로 스트림 처리
.buffered_unordered(32)
.buffered_unordered(16)
```

연결 풀 기반 플로우 컨트롤

연결 수 제한

```
pub struct Options {
pub idle_timeout: Duration,
pub connect_timeout: Duration,
pub max_connections: usize, // 최대 연결 수 제한
```

```
}
```

연결 생명주기 관리

```
async fn run_connection_actor(
self: Arc<Self>,
node_id: NodeId,
mut rx: mpsc::Receiver<RequestRef>,
) {
// 연결 설정
let state = context
.endpoint
.connect(node_id, &context.alpn)
.timeout(context.options.connect_timeout)
.await;
// 유휴 타이머와 작업 처리를 동시에 관리
tokio::select! {
biased;
handler = rx.recv() => {
// 새로운 작업 처리
}
_ = idle_stream.next() => {
// 유휴 상태 처리
}
```

```
_ = &mut idle_timer => {
// 유휴 타임아웃 처리
}
}
}
```

메모리 기반 플로우 컨트롤

청크 단위 처리

```
// 메모리 사용량을 제한하기 위한 청크 단위 처리
let chunk_buf_size = size.min(outboard.tree.block_size().bytes());
let mut buffer = SmallVec::<[u8; 128]>::from_elem(0u8, chunk_buf_size);
```

스마트 벡터 활용

```
// 작은 데이터에 대한 스택 할당 최적화
let mut stack = SmallVec::<[blake3::Hash; 10]>::new();
```

성능 최적화 기법

1. 선택적 처리 (Biased Select)

```
tokio::select! {
```

```
biased; // 우선순위 기반 선택

// 높은 우선순위 작업들...

}
```

2. 스팬 기반 추적

```
fn spawn(&mut self, fut: impl Future<Output = ()> + Send + 'static) {

let span = tracing::Span::current();

let id = self.tasks.spawn(fut.instrument(span)).id();

}
```

3. 효율적인 데이터 구조

- `SmallVec` : 작은 데이터에 대한 스택 할당
- `FuturesUnordered` : 순서에 상관없는 퓨처 처리
- `HashMap` / `HashSet` : 빠른 조회를 위한 해시 기반 컬렉션

오류 처리 및 복구

태스크 실패 처리

```
SelectOutcome::TaskDone(result) => {

if let Err(e) = result {

error!("Task failed: {e:?}");

}
```

```
}
```

연결 실패 복구

```
if  let Err(e) = &state {
debug!(%node_id, "Failed to connect {e:?}, requesting shutdown");
if  context.owner.close(node_id).await.is_err() {
return;
}
}
```

모니터링 및 관찰성

실행 중인 태스크 추적

```
struct DownloaderActor {
running: HashSet<tokio::task::Id>, // 실행 중인 태스크 ID 추적
}
```

구조화된 로깅

```
use tracing::{debug, error, trace, instrument, Instrument};
```

```rust
#[instrument]
async fn handle_connection() {
// 함수 실행이 자동으로 추적됨
}
```

결론

iroh-blob의 동시성 및 비동기 처리 시스템은 다음과 같은 핵심 특징을 가집니다:

1. **계층적 동시성 제어**: 엔티티 매니저를 통한 해시별 독립적 처리
2. **효율적인 태스크 관리**: JoinSet을 활용한 생명주기 관리
3. **자연스러운 백프레셔**: 채널 기반 플로우 컨트롤
4. **리소스 최적화**: 연결 풀링과 액터 재사용
5. **우아한 종료**: 소프트/하드 종료 지원

이러한 설계를 통해 시스템은 높은 처리량과 낮은 지연시간을 달성하면서도 안정적인 동작을 보장합니다.